

DBMS Module 1 — 10 Mark Questions & Answers

Q1. Explain the complete DBMS System Architecture with a neat labeled diagram.

Introduction

A DBMS has a layered architecture separating users, query processing, storage management, and physical data. Understanding this architecture helps in knowing how a user's query travels from input to disk and back.

The Three Types of Users

1. Naive Users:

End-users who interact through application programs. They use pre-built interfaces (forms, apps) without writing SQL. Example: A bank customer withdrawing cash via ATM software.

2. Application Programmers:

Write DML-embedded programs (in Java, Python) that are compiled into object code, which naive users execute. Example: Developer of the ATM application.

3. Sophisticated Users:

Interact directly with the DBMS via SQL clients or report generators. Write ad-hoc queries. Example: Data analysts.

4. Database Administrator (DBA):

The superuser with total control. Defines schemas, manages users, and monitors system health.

The Query Processor

The Query Processor translates user queries into actionable operations:

- **DDL Interpreter:** Parses `CREATE`, `ALTER`, `DROP` statements and records schema changes in the Data Dictionary.
 - **DML Compiler:** Parses and optimizes `SELECT`, `INSERT`, `UPDATE`, `DELETE` queries into an efficient execution plan.
 - **Query Evaluation Engine:** Executes the plan by coordinating with the Storage Manager to fetch or write data.
-

The Storage Manager

The Storage Manager interfaces between the DBMS and the physical disk:

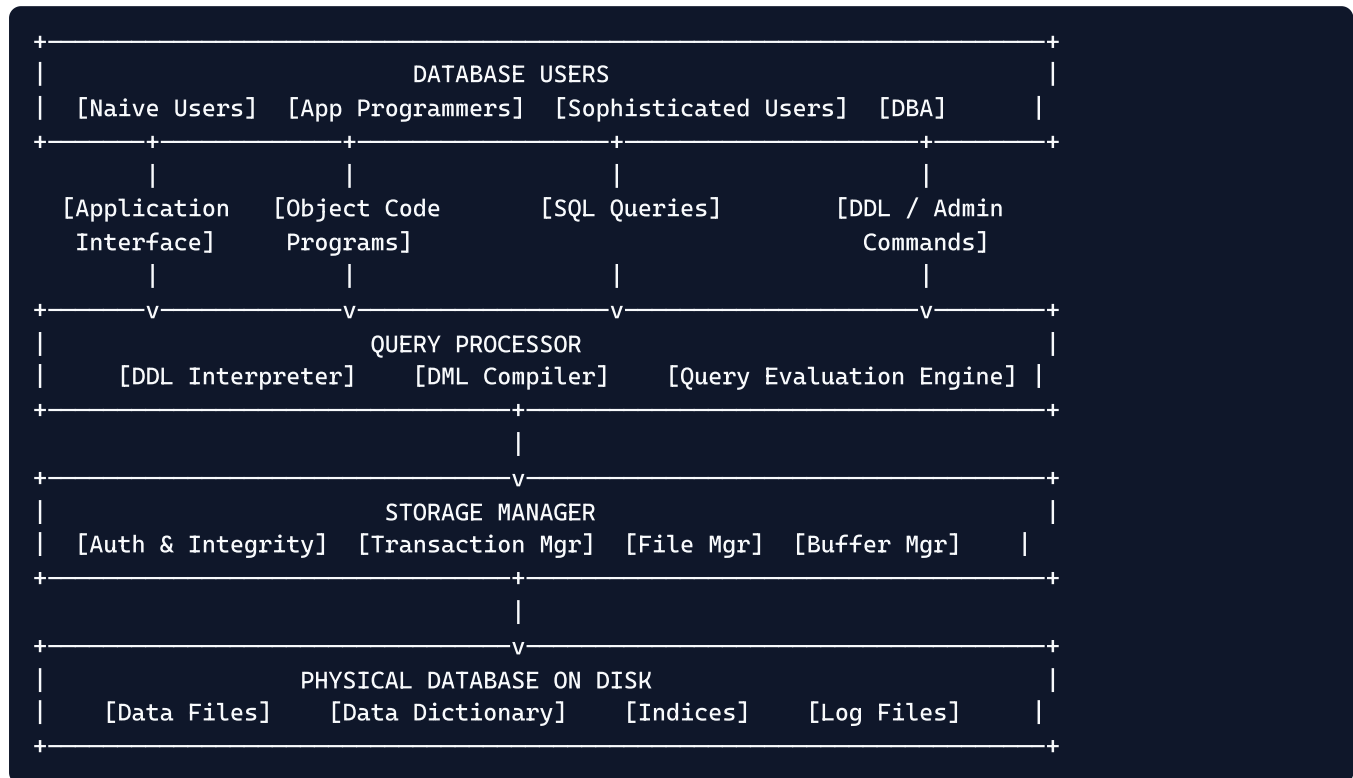
- **Authorization & Integrity Manager:** Validates user access rights and enforces data integrity constraints.
 - **Transaction Manager:** Maintains ACID properties and handles concurrent access via locking protocols.
 - **File Manager:** Manages physical disk space allocation for data files.
 - **Buffer Manager:** Caches frequently accessed data pages in RAM to reduce disk reads.
-

Physical Data Structures

The actual data on disk includes:

- **Data Files:** Store the actual database records.
- **Data Dictionary:** Stores metadata (schema definitions, constraints).
- **Indices:** Speed up data retrieval (e.g., B+ trees).

Complete System Architecture Diagram



Q2. Explain in detail the three-schema architecture and how it achieves data independence.

Introduction

The **Three-Schema Architecture** (also called ANSI/SPARC Architecture) was proposed to provide **data abstraction** and **data independence**. It separates the physical storage, logical design, and user views into three distinct layers, linked by mappings.

The Three Levels in Detail

Level 1 — Internal Level (Physical Schema):

The lowest level, closest to physical storage. Describes how data is actually stored:

- File organization (heap, sequential, clustered)
- Indexing structures (B-tree, hashing)
- Data compression and encryption methods
- Disk block sizes and storage allocation

Managed by: System software and DBA.

Example: Student records stored as a B+ tree indexed file on disk.

Level 2 — Conceptual Level (Logical Schema):

The middle level. Describes the **entire logical structure** of the database visible to the organization:

- All entities and their attributes

- All relationships between entities
- All constraints (primary keys, foreign keys)
- Security and integrity rules

Managed by: DBA.

Example:

```
Student (RollNo INT PK, Name VARCHAR, BranchID INT FK)
Branch (BranchID INT PK, BranchName VARCHAR)
```

Level 3 — External Level (View Level):

The highest level. Each user or user group gets a **customized, partial view** of the database. Multiple external schemas can exist simultaneously. Each view hides irrelevant or unauthorized data.

Managed by: Application programmers / DBA defines views.

Example:

- Faculty sees: StudentView (RollNo, Name, Marks)
- Admin sees: AdminView (RollNo, Name, FeeStatus)

The Two Mappings and Data Independence

Mapping 1: External ↔ Conceptual

Describes how external views relate to the conceptual schema.

- Supports **Logical Data Independence**: If a new column is added to the conceptual schema, only this mapping is updated. The external views and application programs remain unchanged.

Mapping 2: Conceptual ↔ Internal

Describes how the logical schema maps to physical storage.

- Supports **Physical Data Independence**: If the physical storage changes (e.g., indexing method changes), only this mapping is updated. The conceptual schema and all external views are unaffected.

```
[External View 1] [External View 2]
    |
External-Conceptual Mapping ← Updated for Logical Independence
    |
[Conceptual Schema (Full DB)]
    |
Conceptual-Internal Mapping ← Updated for Physical Independence
    |
[Internal Schema (Physical Disk)]
```

Q3. Compare and contrast DBMS Approach and File System Approach in detail.

File System Approach

The **File System** approach stores data in a collection of OS-managed files. Each application program creates and manages its own files independently, with no centralized coordination.

DBMS Approach

The **DBMS** approach stores all data centrally under a dedicated software system that manages storage, access, integrity, and security uniformly for all applications.

Detailed Comparison Table

Feature	File System	DBMS
Data Definition	Data structure defined within application code	Defined separately in schema using DDL
Data Redundancy	Very high; each app maintains its own copy of data	Minimal; centralized storage with normalization
Data Inconsistency	Very common; different files have conflicting values	Eliminated; a single update reflects everywhere
Data Sharing	Very difficult; files belong to specific programs	Easy, controlled sharing across all applications
Data Access / Querying	Custom programs written for each query	Powerful, flexible SQL for ad-hoc queries
Data Security	OS-level file permissions only (coarse-grained)	Fine-grained control (per user, per table, per column)
Data Integrity	No automatic enforcement; programmed manually	DBMS enforces constraints (PK, FK, CHECK, NOT NULL)
Concurrency	No built-in support; simultaneous access corrupts data	Full concurrency control with locking and isolation
Backup & Recovery	Manual backups; no crash recovery	Automatic transaction logs; point-in-time recovery
Data Abstraction	None; physical structure exposed to programs	Three levels of abstraction hide complexity
Data Independence	None; change in file structure breaks all programs	High; changes at one level don't affect others
Cost	Very low initial cost	Higher setup cost; lower long-term maintenance cost
Scalability	Very poor for large, multi-user scenarios	Excellent; designed for large-scale, multi-user access
Example	.csv files, text files, Excel sheets	MySQL, Oracle, PostgreSQL, MongoDB

When to Use Which?

File System: Suitable for very simple, single-user, standalone applications with minimal data — e.g., a personal notes file.

DBMS: Suitable for all real-world, multi-user, enterprise-grade applications — e.g., banking, hospitals, e-commerce, airlines.

Q4. Explain Data Abstraction and Data Independence in detail with diagrams.

Data Abstraction

Data Abstraction is the process of hiding the complex details of how data is stored and maintained, exposing only the information relevant to the user. The goal is to simplify the user's interaction with the database.

The DBMS provides abstraction at **three levels**:

Physical Level — *How is data stored?*

Handles raw storage: file organization (heap, B-trees, hash tables), disk blocks, pointers, compression. The user never sees this.

Logical Level — *What data is stored?*

Defines the tables, columns, data types, and relationships. A programmer creates a table using:

```
CREATE TABLE Employee (  
    EmpID    INT PRIMARY KEY,  
    EmpName  VARCHAR(50),  
    Salary   DECIMAL(10,2)  
);
```

The programmer doesn't need to know if this is a B-tree or a heap file on disk.

View Level — *What does this user see?*

A clerk might see only `EmpID` and `EmpName`, while an HR manager sees `Salary` as well. Multiple views provide security and simplicity.

Data Independence

Logical Data Independence (harder to achieve):

Change the conceptual schema → views and programs remain unaffected.

If the DBA adds a new column `Address` to `Employee`, existing programs that only use `EmpID` and `EmpName` do not break. Only the **External-Conceptual mapping** is updated.

Physical Data Independence (easier to achieve):

Change the physical storage → conceptual schema remains unaffected.

If the DBA switches from a heap file to a clustered B+ tree for the `Employee` table (for faster queries), the SQL schema (`CREATE TABLE`) does not need to change. Only the **Conceptual-Internal mapping** is updated.

```
[View A] [View B]  ← Protected by Logical Data Independence  
|    ↕ Mapping 1 (Updated when logical schema changes)  
[Logical Schema]  ← Protected by Physical Data Independence  
|    ↕ Mapping 2 (Updated when physical schema changes)  
[Physical Storage]
```

Q5. Describe the different categories of Database Users and the role of the DBA.

The Four Categories of Database Users

1. Naive (Parametric) Users:

The largest group. Interact with the database only through carefully designed front-end applications. Their interaction is limited to pre-defined operations (viewing, booking, depositing).

Characteristics:

- No SQL knowledge
- Use forms, menus, buttons
- Cannot modify database structure

Examples: Bank customers, online shoppers, students checking results on a portal.

2. Application Programmers:

Build the applications that naive users interact with. They embed DML calls within programs written in host languages.

Characteristics:

- Know SQL and host programming languages
- Use APIs (JDBC for Java, ODBC, Python connectors)
- Work at the logical level

Examples: Developers building banking apps, hospital management systems.

3. Sophisticated Users:

Directly interact with the DBMS without using predefined programs. They formulate queries using SQL or use business intelligence tools.

Characteristics:

- Strong SQL knowledge
- Can write complex multi-table queries
- Use tools like Power BI, Tableau, or SQL Workbench

Examples: Data analysts, financial consultants, researchers.

4. Specialized Users:

Work with non-traditional or specialized DBMS types for complex applications not suited to standard relational databases.

Examples:

- CAD/CAM engineers using object-oriented databases
- GIS developers using spatial databases
- AI researchers using knowledge-base/graph databases

The Database Administrator (DBA)

The DBA is the most powerful user in the system with complete authority over the database. The DBA's key duties include:

1. **Schema Definition:** Writes the initial DDL to create all tables, views, constraints, and relationships.
2. **Storage Structure and Access Method Definition:** Decides the physical organization, indexing strategy, and partitioning to balance performance and storage cost.
3. **Schema and Physical Organization Modification:** Alters schemas and reorganizes physical storage as business requirements evolve, without disrupting ongoing operations.

4. **Granting User Authorizations:** Uses `GRANT` and `REVOKE` to control who can read, insert, update, or delete specific data. Implements role-based access control (RBAC).
 5. **Routine Maintenance:** Schedules and performs regular data backups. Tests restore procedures. Monitors disk usage and query performance.
 6. **Performance Tuning:** Analyzes slow queries using execution plans, creates or drops indexes, restructures queries, and adjusts buffer sizes for optimal throughput.
 7. **Database Security Auditing:** Reviews audit logs for suspicious access patterns or policy violations. Implements encryption for sensitive data at rest and in transit.
-

Q6. Explain the Query Processor and Storage Manager and how a query flows from user to disk.

Overview

When a user submits a SQL query, it travels through multiple layers of the DBMS before the result is returned. This flow involves both the **Query Processor** and the **Storage Manager**.

The Query Processor Components

DDL Interpreter:

Handles `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`. Parses the statement, validates it against existing schema, and if valid, records the new/changed schema into the **Data Dictionary**.

DML Compiler:

Handles `SELECT`, `INSERT`, `UPDATE`, `DELETE`.

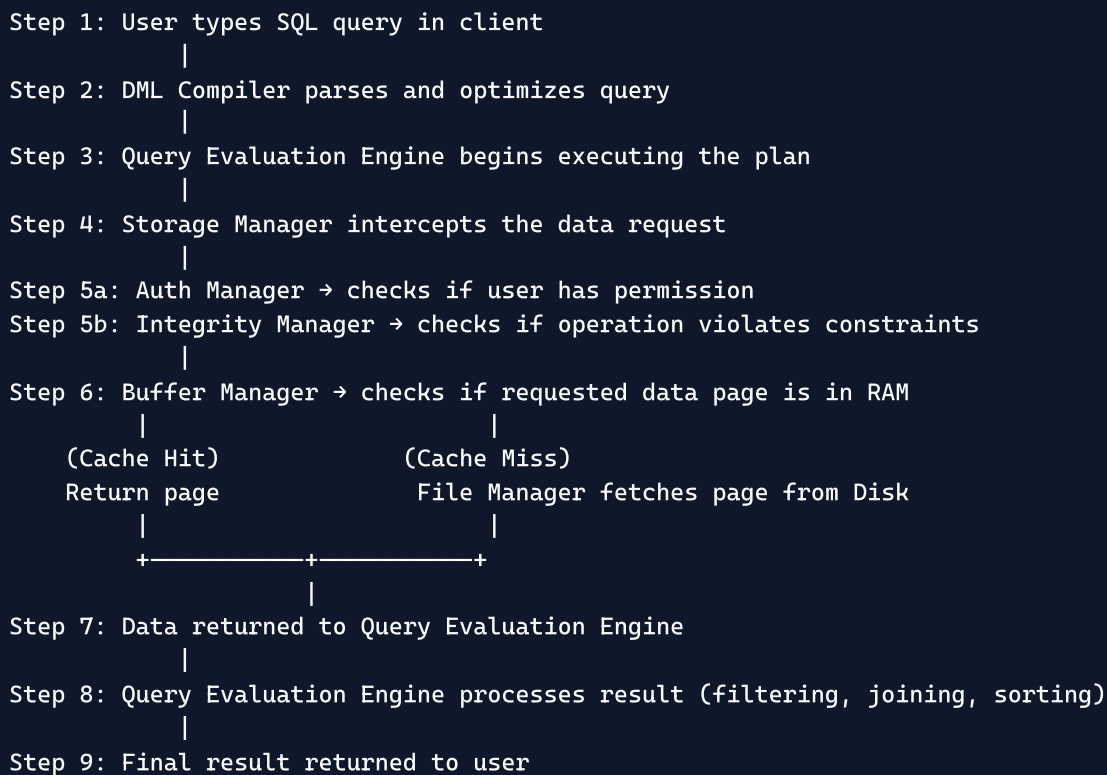
Steps performed:

1. **Parsing:** Checks syntax and semantics (is the table name valid? Do columns exist?).
2. **Query Optimization:** Generates multiple possible query plans (e.g., should it use an index or a full scan? Which table to join first?). Chooses the most efficient plan based on statistics in the Data Dictionary.
3. **Plan Generation:** Produces a low-level execution plan as a query execution tree.

Query Evaluation Engine:

Executes the query plan step by step. Issues read/write requests to the **Storage Manager** to fetch or modify actual data on disk.

Complete Query Flow



Q7. Discuss the need for a database system over traditional file-based systems.

Introduction

Before DBMS, organizations stored data in operating system files (text files, CSV files). While simple to create, this approach caused numerous serious problems as data grew in size and importance.

Advantages of DBMS / Why Database is Needed

1. Controlling Data Redundancy:

In file systems, the same student's address might exist in the payroll file, the enrollment file, and the library file. If the address changes, all three must be updated manually — and if one is missed, data is inconsistent. A DBMS centralizes storage, so each data item exists in one place only.

2. Restricting Unauthorized Access:

File systems only provide basic OS-level read/write/execute permissions on files. A DBMS provides fine-grained, role-based access control — e.g., an accounts clerk can update salary records but cannot view medical records.

3. Providing Persistent Storage:

Programs like calculators lose data when closed. A DBMS provides **persistent storage**, ensuring that all data committed to the database survives program crashes and system reboots.

4. Storage Structures for Efficient Query Processing:

A DBMS creates indexes, buffers, and other data structures to retrieve data extremely quickly. A flat file requires reading every line to find one record; a DBMS uses a B+ tree index to find it in milliseconds.

5. Providing Backup and Recovery:

Hard drive failures, power outages, or software bugs can destroy file-based data permanently. A DBMS

automatically logs every transaction and can recover the database to a consistent state after any failure — critical for banking and hospital systems.

6. Providing Multiple User Interfaces:

Different users need different interaction methods. A DBMS supports SQL for analysts, graphical forms for clerks, APIs for programmers, and touch interfaces for end customers — all accessing the same underlying data.

7. Representing Complex Relationships:

Real-world data is inter-related. A student belongs to a department, takes multiple courses, and has multiple marks. File systems cannot natively model these **many-to-many** and **one-to-many** relationships. A DBMS handles complex relationships elegantly using foreign keys and joins.

8. Enforcing Integrity Constraints:

Data quality is critical. A DBMS ensures that a student's marks cannot be -10 or 200 (via CHECK constraints), that every order must refer to a valid customer (via FOREIGN KEY), and that no two students share the same roll number (via PRIMARY KEY).

9. Concurrency Control:

When thousands of users simultaneously book airline tickets or transfer money, file systems corrupt data through race conditions. A DBMS's transaction management ensures that simultaneous operations are isolated, maintaining consistency.

Q8. Explain DBMS architecture types (1-tier, 2-tier, 3-tier) in detail with diagrams, advantages/disadvantages, and use cases.

Introduction

DBMS architecture defines how the various components of a database system are distributed across different machines (tiers/layers). Choosing the right architecture is critical for performance, scalability, and maintainability.

1-Tier Architecture

Definition: All components — user interface, application logic, and database — reside on the **same single machine**. The user directly interacts with the database without any network.

Diagram:



Advantages:

- Simple to set up and use

- Fast (no network latency)
- Low cost (no server needed)

Disadvantages:

- Not scalable (single user only)
- No data sharing
- Security is poor (all access on one machine)

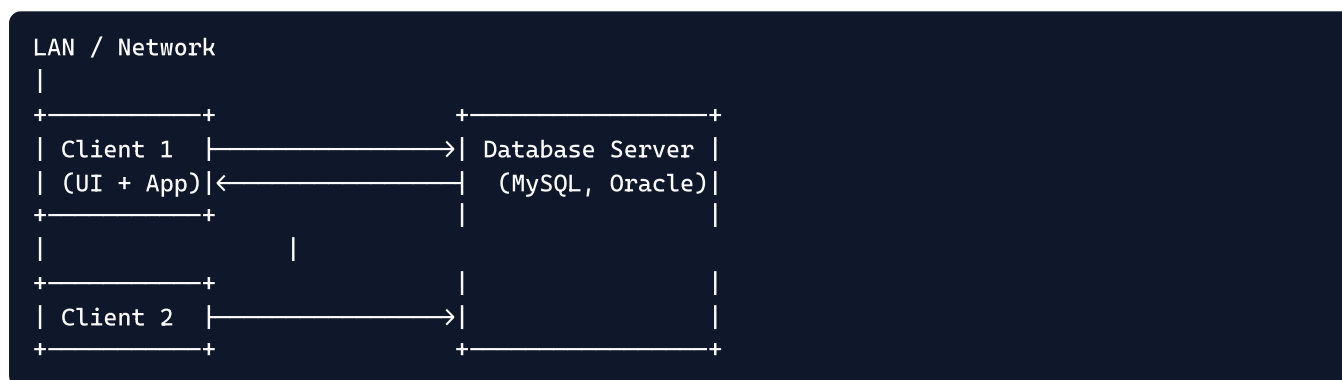
Use Case: Personal databases, local development environments, simple standalone tools.

Example: Microsoft Access, SQLite in a personal app.

2-Tier Architecture

Definition: Splits the system into two tiers — the **client** (presentation + application logic) and the **database server** (data storage). They communicate over a network.

Diagram:



Advantages:

- Supports multiple users simultaneously
- Better performance than 1-tier for small networks
- Easier than 3-tier to set up

Disadvantages:

- Business logic is embedded in client — hard to maintain and update
- Poor scalability for internet-level users
- Security risk: client directly connects to DB server

Use Case: Small office networks, LAN-based internal business applications.

Example: A school's desktop fee management system connecting to a shared school server.

3-Tier Architecture

Definition: The most modern and widely-used architecture. Splits the system into **three distinct and independent layers**: Presentation Tier, Application Tier, and Database Tier.

Diagram:



Advantages:

- Highly scalable — each tier can be scaled independently
- High security — clients never directly access the database
- Easy maintenance — changing business logic only affects Tier 2
- Multiple clients (web, mobile, desktop) can use the same Tier 2 and Tier 3

Disadvantages:

- More complex to design, deploy, and debug
- Higher infrastructure cost (multiple servers)

Use Case: All modern web and mobile applications at scale.

Example: Online banking, e-commerce websites (Amazon, Flipkart), hospital management systems.

Summary Comparison

Feature	1-Tier	2-Tier	3-Tier
Layers	1	2	3
Users	Single	Limited (LAN)	Unlimited (Internet)
Scalability	Very Low	Medium	Very High
Security	Low	Medium	High
Complexity	Very Low	Medium	High
Cost	Very Low	Medium	High
Example	SQLite local app	School LAN system	Amazon, Banking apps

Q9. Explain the Entity-Relationship Model in detail — entities, entity sets, relationship sets, attributes — with notations and examples.

Introduction

The **Entity-Relationship (ER) Model** is a high-level conceptual data model developed by Peter Chen in 1976. It allows database designers to express the logical structure of a database graphically using entities, attributes, and relationships.

1. Entities and Entity Sets

- **Entity:** A distinguishable object in the real world (e.g., Student "Alice", Course "DBMS").
- **Entity Set:** A collection of entities of the same type sharing common properties.
- **Strong Entity Set:** Possesses a primary key. (Drawn as a **Single Rectangle**).
- **Weak Entity Set:** Lacks a primary key; depends on an owner entity. (Drawn as a **Double Rectangle**).

2. Attributes

Properties describing an entity set:

- * **Simple:** Atomic (e.g., Age).
- * **Composite:** Multi-part (e.g., Name → First_Name , Last_Name).
- * **Single-valued:** Exactly 1 value (e.g., DOB).
- * **Multivalued:** Multiple values (e.g., Phone_Number). (Drawn as **Double Ellipse**).
- * **Derived:** Computed dynamically (e.g., Age). (Drawn as **Dashed Ellipse**).
- * **Key Attribute:** Uniquely identifies entity. (Drawn with **Underlined Text**).

3. Relationship Sets

An association among entities from two or more entity sets.

- * **Degree:** Number of participating entity sets (Binary, Ternary, N-ary).
- * **Descriptive Attributes:** Attributes attached to the relationship itself (e.g., Grade in TAKES).
- * **Mapping Cardinalities:** 1:1, 1:N, N:1, M:N.
- * **Participation:** Total (Double Line) or Partial (Single Line).

ER Symbol Notation Chart

Entity Set +-----+ EMPLOYEE +-----+	Weak Entity Set +=====+ DEPENDENT +=====+	Attribute /-----\ (Salary) \-----/
Key Attribute /-----\ (Emp_ID) \-----/ _____	Multivalued Attr /=====\ ((Phone_Number)) \=====/ _____	Derived Attr /-----\ (Age) \-----/
Relationship ^ / \ \ ^	Identifying Rel ≠\ // \ \\ // \\ // ≠/	Total Participation =====
		Partial Participation _____

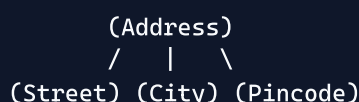
Q10. Explain all types of Attributes with diagrams and real examples.

Introduction

An **Attribute** is a property or characteristic that describes an entity in an entity set or a relationship in a relationship set.

1. Simple vs. Composite Attributes

- **Simple Attribute:** Cannot be sub-divided into smaller components. Example: `Emp_ID` , `Salary` .
- **Composite Attribute:** Can be sub-divided into smaller sub-attributes forming a hierarchy. Example: `Address` composite attribute breaks into `Street` , `City` , `State` , `Pincode` .



2. Single-Valued vs. Multivalued Attributes

- **Single-Valued Attribute:** Holds a single value for an entity instance. Example: `Date_of_Birth` , `Gender` .
- **Multivalued Attribute:** Holds multiple values for a single entity instance. Represented as a **Double Ellipse**. Example: `Email_Address` (an employee can have personal and work emails).

3. Stored vs. Derived Attributes

- **Stored Attribute:** Saved physically on disk. Example: `Date_of_Birth` .
- **Derived Attribute:** Not stored on disk; calculated dynamically from stored data when queried. Represented as a **Dashed Ellipse**. Example: `Age = Current_Date - Date_of_Birth` .

4. Key Attributes vs. Null Attributes

- **Key Attribute:** Uniquely identifies each entity in an entity set. Text is **Underlined**. Example: `Roll_No` .
- **Null Value:** Used when an attribute value is missing, unknown, or not applicable. Example: `Middle_Name` for a person without one.

Q11. Explain Mapping Cardinality Constraints in detail (1:1, 1:N, N:1, M:N), including min..max notation and Total/Partial participation.

1. Mapping Cardinalities

1. One-to-One (1:1):	2. One-to-Many (1:N):	3. Many-to-Many (M:N):
[A1] $\longleftrightarrow$ [B1]	[A1] $--+\longrightarrow$ [B1]	[A1] $--+\longrightarrow$ [B1]
[A2] $\longleftrightarrow$ [B2]	$+\longrightarrow$ [B2]	X
	[A2] $\longrightarrow$ [B3]	[A2] $--+\longrightarrow$ [B2]

- **One-to-One (1:1):** An entity in A maps to at most 1 entity in B, and vice versa. Example: `EMPLOYEE` *manages* `DEPARTMENT` .

- **One-to-Many (1:N):** An entity in A maps to many entities in B, but B maps to at most 1 in A. Example: `DEPARTMENT employs EMPLOYEE` .
- **Many-to-One (N:1):** Many entities in A map to 1 entity in B. Example: `STUDENT enrolled in DEPARTMENT` .
- **Many-to-Many (M:N):** An entity in A maps to many in B, and vice versa. Example: `STUDENT takes COURSE` .

2. Min..Max Constraint Notation

Replaces basic cardinalities with explicit structural constraints `(min..max)` placed on participating edges:

* **min:** Minimum number of relationships an entity must participate in ($\text{min} = 0 \rightarrow \text{Partial}$, $\text{min} \geq 1 \rightarrow \text{Total}$).

* **max:** Maximum number of relationships an entity can participate in.

Example: `EMPLOYEE (0..1) — MANAGES — (1..1) DEPARTMENT`

- Employee can manage 0 or 1 department.

- Department MUST have exactly 1 manager.

Q12. Explain Weak and Strong Entity Sets in detail — discriminator, identifying relationship, notation, and a complete example with primary key derivation.

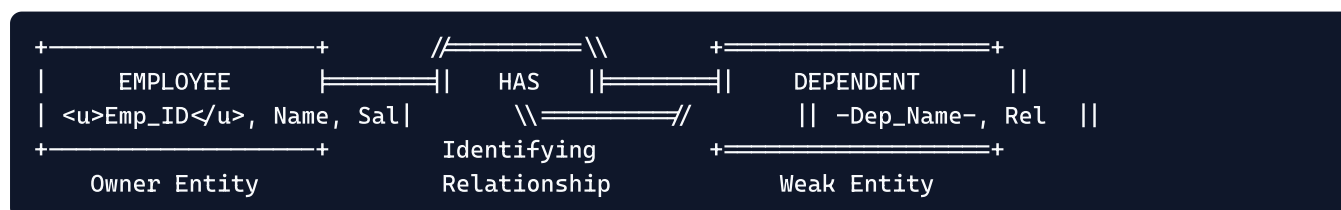
1. Concept

- **Strong Entity Set:** Has a primary key sufficient to uniquely identify all entities.
- **Weak Entity Set:** Cannot be uniquely identified by its own attributes alone. It depends on a strong **Owner Entity Set** via an **Identifying Relationship Set**.

2. Discriminator (Partial Key)

A weak entity set has a **Discriminator** (or Partial Key) — an attribute that distinguishes entities belonging to the *same* owner entity. Represented with a **dashed underline**.

3. Complete Example: `EMPLOYEE` & `DEPENDENT`



- **Owner Entity:** `EMPLOYEE(Emp_ID, Name, Salary)` → `PK = Emp_ID` .
- **Weak Entity:** `DEPENDENT(Dep_Name, Relationship, Birth_Date)` → `Discriminator = Dep_Name` .
- **Identifying Relationship:** `HAS` (Double Diamond, double line to `DEPENDENT`).
- **Primary Key Derivation for `DEPENDENT`:**
Primary Key of `DEPENDENT` = { `Emp_ID` (Owner PK), `Dep_Name` (Discriminator) }

Q13. Explain Keys in the ER model in detail — primary key for entity sets and relationship sets, and key rules for all four binary relationship types.

1. Types of Keys

- **Superkey:** A set of one or more attributes that uniquely identifies an entity.
 - **Candidate Key:** A minimal superkey (no redundant attributes).
 - **Primary Key:** The candidate key selected by the database designer.
-

2. Key Rules for Relationship Sets

Let R be a binary relationship set between entity sets A and B:

1. Many-to-Many (M:N):

- * Primary Key of R = **Union of Primary Keys of A and B**.
- * $PK(R) = (PK_A, PK_B)$.

2. One-to-Many (1:N):

- * Primary Key of R = **Primary Key of the "Many" side (B)**.
- * $PK(R) = PK_B$.

3. Many-to-One (N:1):

- * Primary Key of R = **Primary Key of the "Many" side (A)**.
- * $PK(R) = PK_A$.

4. One-to-One (1:1):

- * Primary Key of R = **Primary Key of ANY ONE entity set** (PK_A or PK_B).
-

Q14. Explain Specialization and Generalization in detail — overlapping/disjoint, attribute inheritance, single/multiple inheritance (lattice), completeness constraints.

1. Definitions

- **Specialization (Top-Down):** Splitting a superclass into sub-groupings based on specific traits.
 - **Generalization (Bottom-Up):** Combining multiple subclasses into a single superclass based on shared traits.
-

2. Constraints on Hierarchies

- **Disjointness Constraint:**
 - **Disjoint ('d'):** Entity belongs to at most 1 subclass (e.g., EMPLOYEE is SALARIED or HOURLY).
 - **Overlapping ('o'):** Entity can belong to multiple subclasses (e.g., PERSON is STUDENT and EMPLOYEE).
 - **Completeness Constraint:**
 - **Total (Double Line):** Every superclass entity MUST belong to a subclass.
 - **Partial (Single Line):** Some superclass entities do not belong to any subclass.
-

3. Inheritance & Lattice Structure

- **Attribute Inheritance:** Subclasses inherit all attributes and relationships of the superclass.
- **Single Inheritance (Tree):** Subclass has exactly 1 superclass.
- **Multiple Inheritance (Lattice):** A subclass inherits from **more than 1 superclass** (e.g., TEACHING_ASSISTANT inherits from both STUDENT and EMPLOYEE).

Q15. Explain Aggregation in the EER model in detail with the proj_guide/eval_for example — why it's needed and how it's represented.

Why Aggregation is Needed

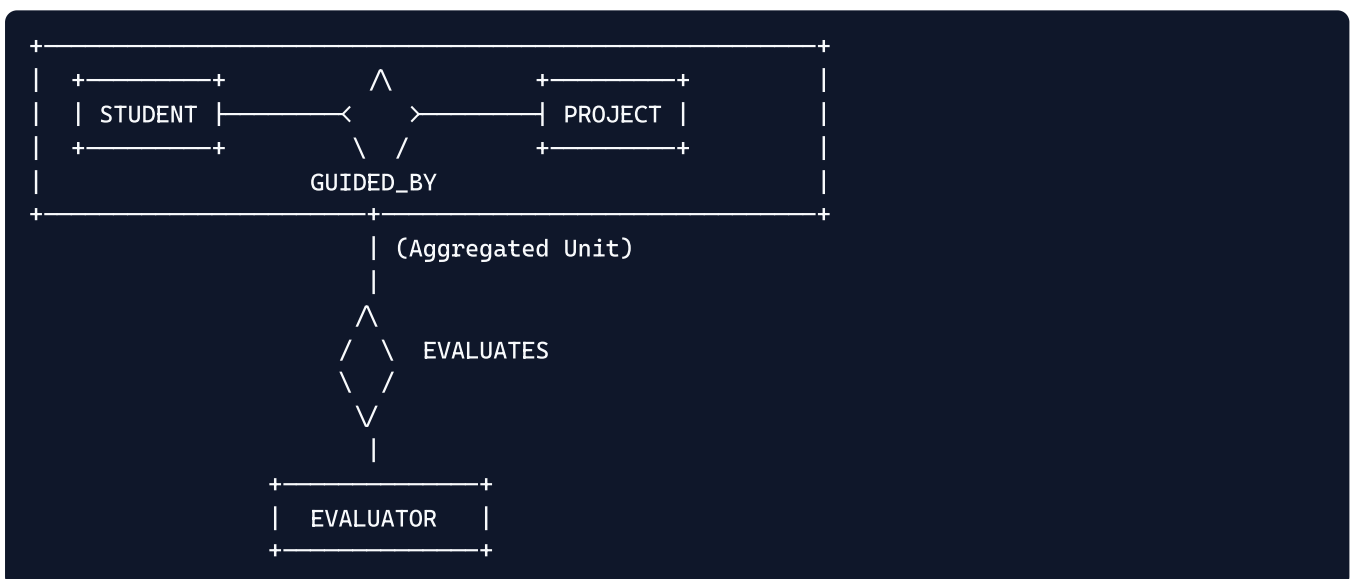
Standard ER modeling prohibits relationships between relationships. If we need to express a relationship that involves another relationship, standard ER fails or produces incorrect ternary semantics.

Example Scenario

- STUDENT works on PROJECT via relationship GUIDED_BY (with INSTRUCTOR).
- Now, an EVALUATION relationship must record the MARKS awarded to the entire [STUDENT – GUIDED_BY – PROJECT] activity by an EVALUATOR.

Diagram & Representation

Aggregation places a bounding box around [STUDENT, GUIDED_BY, PROJECT], treating it as a single higher-level entity set that connects to EVALUATOR via EVALUATES.



Q16. Draw and explain a complete ER diagram for a University Database System.

System Requirements

- **Entities:** STUDENT, INSTRUCTOR, COURSE, DEPARTMENT, SECTION.
- **Weak Entity:** SECTION (dependent on COURSE).
- **Multivalued Attribute:** Phone_Number on INSTRUCTOR.

Relational Schema Derived from ER

1. DEPARTMENT (Dept Name, Building, Budget)
2. INSTRUCTOR (ID, Name, Salary, Dept_Name)
3. INSTRUCTOR_PHONE (ID, Phone Number)

4. STUDENT (ID, Name, Tot_Cred, Dept_Name)
 5. COURSE (Course_ID, Title, Credits, Dept_Name)
 6. SECTION (Course_ID, Sec_ID, Semester, Year, Building, Room_Number)
 7. TEACHES (ID, Course_ID, Sec_ID, Semester, Year)
 8. TAKES (ID, Course_ID, Sec_ID, Semester, Year, Grade)
-

Q17. Explain all Constraints in the ER Model — Key Constraint, Participation Constraint, and Mapping Cardinality Constraint.

1. **Key Constraint:** Specifies that a set of attributes uniquely identifies entities in an entity set or limits an entity to at most 1 relationship instance (1:1 or 1:N).
 2. **Participation Constraint:**
 - * **Total:** Every entity MUST participate (Double Line).
 - * **Partial:** Optional participation (Single Line).
 3. **Mapping Cardinality Constraint:** Limits the number of entities related via a relationship set (1:1, 1:N, N:1, M:N).
-

Q18. Design an ER diagram for the Employee-Department-Project-Dependent scenario.

Entity Sets & Attributes

- EMPLOYEE (Emp_ID, Name, Salary, DOB)
 - DEPARTMENT (Dept_ID, Dept_Name, Location)
 - PROJECT (Proj_ID, Proj_Name, Budget)
 - DEPENDENT (Dep_Name [Partial Key], Sex, Relationship) [Weak Entity]
-

Relationships

- WORKS_IN (EMPLOYEE N : 1 DEPARTMENT) [Total participation for Employee]
 - MANAGES (EMPLOYEE 1 : 1 DEPARTMENT) [Partial for Employee, Total for Dept]
 - WORKS_ON (EMPLOYEE M : N PROJECT) [With attribute Hours]
 - HAS_DEPENDENT (EMPLOYEE 1 : N DEPENDENT) [Identifying relationship for weak entity]
-

Q19. Draw a complete ER diagram for the Company Database; convert it fully into a relational schema with keys.

Converted Relational Schema

1. EMPLOYEE (Emp_ID, FName, LName, BDate, Address, Sex, Salary, Super_Emp_ID, Dept_ID)
 - * Super_Emp_ID references EMPLOYEE(Emp_ID)
 - * Dept_ID references DEPARTMENT(Dept_ID)
2. DEPARTMENT (Dept_ID, Dept_Name, Mgr_Emp_ID, Mgr_Start_Date)
 - * Mgr_Emp_ID references EMPLOYEE(Emp_ID)
3. DEPT_LOCATIONS (Dept_ID, Location)
 - * Dept_ID references DEPARTMENT(Dept_ID)

4. PROJECT (Proj_ID, Proj_Name, Location, Dept_ID)
* Dept_ID references DEPARTMENT(Dept_ID)
 5. WORKS_ON (Emp_ID, Proj_ID, Hours)
* Emp_ID references EMPLOYEE(Emp_ID)
* Proj_ID references PROJECT(Proj_ID)
 6. DEPENDENT (Emp_ID, Dep_Name, Sex, BDate, Relationship)
* Emp_ID references EMPLOYEE(Emp_ID)
-

Q20. Explain all eight steps of mapping an ER diagram to a relational model, with one example per step.

- **Step 1: Strong Entity Sets** → Create relation with all simple attributes.
 - **Step 2: Weak Entity Sets** → Create relation including Owner PK + Discriminator as PK.
 - **Step 3: Binary 1:1 Relationship** → Include PK of one side as FK in the total participation side.
 - **Step 4: Binary 1:N Relationship** → Include PK of "1" side as FK in the "N" side table.
 - **Step 5: Binary M:N Relationship** → Create new junction table with PKs of both sides as composite PK.
 - **Step 6: Multivalued Attributes** → Create new table (Owner_PK, Multivalued_Attr) .
 - **Step 7: N-ary Relationships** → Create new table with PKs of all participating entity sets.
 - **Step 8: Specialization/Generalization** → Map using one of the 4 specialization mapping options.
-

Q21. Choose two systems (Railway Reservation and Online Food Delivery) — draw ER diagrams and derive relational schemas for both.

System 1: Railway Reservation System

- **Relations:**
 1. PASSENGER (PNR, Name, Age, Gender, Contact)
 2. TRAIN (Train_No, Train_Name, Source, Destination)
 3. TICKET (Ticket_No, PNR, Train_No, Seat_No, Booking_Date, Fare)

System 2: Online Food Delivery System

- **Relations:**
 1. CUSTOMER (Cust_ID, Name, Phone, Address)
 2. RESTAURANT (Rest_ID, Name, City, Rating)
 3. FOOD_ITEM (Item_ID, Rest_ID, Name, Price)
 4. ORDERS (Order_ID, Cust_ID, Rest_ID, Order_Date, Total_Amount)
-

Q22. Design a complete ER diagram for the University Database (Student, Course, Instructor, Department, Exam) with a weak entity and multivalued attribute; convert fully to relational schema.

Relational Schema

1. DEPARTMENT (Dept_Code, Dept_Name, Building)
2. INSTRUCTOR (Inst_ID, Name, Salary, Dept_Code)
3. INST_EXPERT_AREA (Inst_ID, Expertise) [Multivalued]

4. STUDENT (Roll_No, Name, GPA, Dept_Code)
 5. COURSE (Course_Code, Title, Credits, Dept_Code)
 6. EXAM (Course_Code, Exam_ID, Exam_Date, Max_Marks) [Weak entity dependent on COURSE]
 7. ENROLLMENT (Roll_No, Course_Code, Semester, Grade) [M:N Relationship]
-

Q23. Explain specialization/generalization mapping using

EMPLOYEE→SECRETARY/TECHNICIAN/ENGINEER and VEHICLE→CAR/TRUCK — show all four mapping approaches and discuss trade-offs.

Example 1: EMPLOYEE → SECRETARY , TECHNICIAN , ENGINEER

- **Option 1 (Superclass + Subclass tables):**

- EMPLOYEE(Emp_ID, Name, Address, Job_Type)
- SECRETARY(Emp_ID, Typing_Speed)
- TECHNICIAN(Emp_ID, TGrade)
- ENGINEER(Emp_ID, Eng_Type)
- *Trade-off:* Clean, no NULLs, but requires JOINS to retrieve complete subclass data.

- **Option 2 (Subclass-only tables - for Total/Disjoint):**

- SECRETARY(Emp_ID, Name, Address, Typing_Speed)
- TECHNICIAN(Emp_ID, Name, Address, TGrade)
- ENGINEER(Emp_ID, Name, Address, Eng_Type)
- *Trade-off:* Fast queries on single subclass, but duplicates superclass attribute definitions.

- **Option 3 (Single relation with 1 Type attribute):**

- EMPLOYEE(Emp_ID, Name, Address, Job_Type, Typing_Speed, TGrade, Eng_Type)
- *Trade-off:* Zero JOINS required, but contains many NULL values for non-applicable columns.

- **Option 4 (Single relation with Multiple Type attributes - for Overlapping):**

- EMPLOYEE(Emp_ID, Name, Address, Is_Sec, Is_Tech, Is_Eng, Typing_Speed, TGrade, Eng_Type)
 - *Trade-off:* Supports overlapping roles in 1 table, but wastes memory with NULLs.
-